SQL Backend Data Validation & QA Testing Portfolio

Name: Devana Srinivas Reddy
Role: Junior QA Tester
Focus: Manual Testing, SQL Backend Validation

Project Overview

Project Name: Transactional Data Integrity Validation using SQL
Objective: Simulate backend testing scenarios by validating relational data consistency between users, orders, and payments tables.

Technologies Used

SQL

Relational Database Concepts

Manual Testing Methodologies

Data Validation Techniques

Database Schema

Users Table:
CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(100), city VARCHAR(50));

Orders Table:
CREATE TABLE orders (order_id INT PRIMARY KEY, user_id INT, amount INT);

Payments Table:
CREATE TABLE payments (payment_id INT PRIMARY KEY, order_id INT, payment_amount INT);

Test Scenarios

Scenario 1	Detect duplicate user registrations
Scenario 2	Identify orders without payment records
Scenario 3	Verify financial consistency between orders and payments
Scenario 4	Detect invalid order amounts
Scenario 5	Identify high value customers using aggregation queries

SQL Validation Queries

Duplicate User Detection:
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

Orders Without Payment Records:
SELECT orders.order_id FROM orders LEFT JOIN payments ON orders.order_id = payments.order_id WHERE payments.order_id IS NULL;

Financial Consistency Validation:
SELECT users.id, SUM(orders.amount) AS order_total, SUM(payments.payment_amount) AS payment_total FROM users JOIN orders ON users.id = orders.user_id JOIN payments ON orders.order_id = payments.order_id GROUP BY users.id HAVING SUM(orders.amount) != SUM(payments.payment_amount);

Test Case Examples

Test Case ID	Test Scenario	Expected Result	Status
TC001	Duplicate users	No duplicate email allowed	Fail
TC002	Orders without payment	Every order must have payment	Fail
TC003	Payment reconciliation	Order total equals payment total	Pass

Bug Report Example

Bug ID: BUG_001
Title: Order created without payment record
Severity: High

Steps to Reproduce:
1. Join orders table with payments table
2. Filter orders without payment records

SQL Query:
SELECT orders.order_id FROM orders LEFT JOIN payments ON orders.order_id = payments.order_id WHERE payments.order_id IS NULL;

Expected Result: Every order should have a corresponding payment.
Actual Result: Order ID 105 has no payment record.

Test Execution Summary

Metric	Value
Total Test Cases	5
Passed	3
Failed	2

Detected Issues

Missing payment record

Duplicate user registration

Conclusion

This project demonstrates backend QA validation using SQL. The work validates relational data integrity, financial consistency, and transaction accuracy across database tables.